

A overview of binaries, ELF, and NoMMU on Linux (landley.net)

125 points by [oliverkwebb](#) 10 months ago | [unvote](#) | [hide](#) | [past](#) | [favorite](#) | [4 comments](#)

[matheusmoreira](#) 10 months ago | [next](#) [-]

That's a *lot* of information in a single email! Really nice, especially the history, virtual memory and nommu stuff. I recently implemented an ELF I/O library and was forced to learn a not insignificant fraction of all that from the specification and scattered resources... An email like this would have made things so much easier.

[melchizedek6809](#) 10 months ago | [prev](#) [-]

> Some old processors had page fault handling in hardware. This sucked rocks, and Linus yelled at the PowerPC guys somewhat extensively about this 20 years ago.

Wait, wasn't it the other way around? I might be mistaken but wasn't one of the problems with PowerPC that it only really had a TLB and the kernel had to walk the page tables in software?

Afaik on x86 the page fault handler is only called when a page isn't marked present, so that one can allocate a new page/load the page from mass storage, but apart from that walking the page tables is done in hardware.

Has been a while since I've only really dabbled in 32-bit protected mode a decade or so ago so I might be misremembering.

[kijiki](#) 10 months ago | [parent](#) [-]

MIPS had a fully classic RISC MMU, TLB only. TLB miss resulted in a fault which the OS handled to walk whatever data structure they chose for memory mapping. They had the KSEG regions which let the (virtual mapped) kernel easily access physical mapped memory for the walk. Not sure if this changed with MIPS64, though KSEGs would have been much less costly in terms of address space there.

PPC (at least Book-E variants) had a more complicated setup where TLB misses did a hash table lookup in HW. If that missed as well, it faulted to the kernel to do the full walk. The trick PPC used was that the page fault handler ran with paging disabled entirely, so it could access physical memory directly while handling the miss, no KSEGs necessary.

No idea how SPARC handled this, but x86/x86-64/ARM all do this entirely in hardware, though in practice it is really microcode.

[namibj](#) 10 months ago | [root](#) | [parent](#) [-]

Can you provide some citation for the claim that x86-64 (assuming something modern like AMD Zen or Intel (post) Skylake P-core) does page table walking/TLB-filling in microcode instead of the fairly obvious state machine that can walk as quickly as the cache hierarchy can deliver the table entries? Well, maybe give it a full cycle latency to process the response and decide the next step, though I don't remember there being any addition required to generate the address of the next level's page table entry so the bit of combinatorics to control the cache's read port might fit in the margins between the port's data out latches becoming valid and the address in latches's setup deadline.

Consider applying for YC's Fall 2025 batch! Applications are open till Aug 4

[Guidelines](#) | [FAQ](#) | [Lists](#) | [API](#) | [Security](#) | [Legal](#) | [Apply to YC](#) | [Contact](#)

Search: